

Design Justification Report

| INT8 MAC Accelerator for CNN Conv2D Acceleration

ECE 410/510 Spring 2026 | Devdutt Mallick

Submitted: June 2026

| 1. Problem and Motivation

Convolutional neural networks are the dominant architecture for image classification, object detection, and feature extraction. The inner loop of every CNN is the Conv2D operation, which repeatedly computes a dot product between a learned weight kernel and a patch of input activations. On general-purpose CPUs this operation is memory-intensive: the processor must fetch weight and activation data from DRAM or cache for every multiply-accumulate operation, and the data reuse pattern of a sliding window convolution does not map efficiently onto the hardware prefetcher.

To quantify this, the `Conv2D._im2col` kernel of a custom NumPy CNN was profiled using Python cProfile on an Intel i5-1135G7 laptop. The profiler recorded 228 calls to this kernel over a complete forward pass, with a total execution time of 0.102 seconds and an average of 0.447 milliseconds per call. This kernel was identified as the dominant contributor to inference latency. Using the roofline model, the measured arithmetic intensity of this kernel is 1.057 FLOP/byte, which sits below the ridge point of 5.25 FLOP/byte on the i5-1135G7.

The kernel is memory-bound on this platform: the CPU is spending time waiting for data rather than computing.

The motivation for custom hardware is to escape this bottleneck. A dedicated INT8 MAC accelerator moves the computation to a streaming architecture where data arrives through a high-bandwidth interface and is consumed immediately by the compute engine, eliminating the overhead of the general-purpose memory hierarchy. The target platform is the sky130A open-source PDK, which provides a reproducible, tool-accessible path from RTL to layout without proprietary silicon access.

| 2. Roofline Analysis

The roofline model provides the theoretical framework for this design. Two platform rooflines were analyzed:

Intel i5-1135G7 (software baseline):

Peak compute: 268.8 GFLOP/s. Memory bandwidth: 51.2 GB/s.

Ridge point: $268.8 / 51.2 = 5.25$ FLOP/byte.

The Conv2D._im2col kernel at $AI = 1.057$ FLOP/byte is to the left of the ridge point, confirming it is memory-bound on this platform.

sky130A ASIC (accelerator target):

Peak compute: 0.2 GFLOP/s (100 MHz, 1 MAC/cycle, 2 FLOP/MAC).

Interface bandwidth: 3.2 GB/s (AXI4-Stream 128-bit at 200 MHz).

Ridge point: $0.2 / 3.2 = 0.063$ FLOP/byte.

The kernel's arithmetic intensity range was computed as two bounds: $AI_{low} = 0.82$ FLOP/byte (no data reuse, loading all 9 weights and 9 activations fresh per invocation) and $AI_{high} = 1.38$ FLOP/byte (weight-stationary reuse across a 224×224 output map). Both values fall to the right of the sky130A ridge point. This means the accelerator is compute-bound on its own roofline: the interface can deliver data faster than the single MAC core can consume it.

This analysis shaped the architecture in two ways. First, a streaming interface (AXI4-Stream rather than a memory-mapped register file) was chosen because the interface bandwidth is not the bottleneck — a low-overhead stream protocol maximizes data delivery to the compute engine. Second, the identified compute bottleneck motivates the M4 roadmap: the highest-leverage improvement is parallelizing the MAC array, not widening the interface.

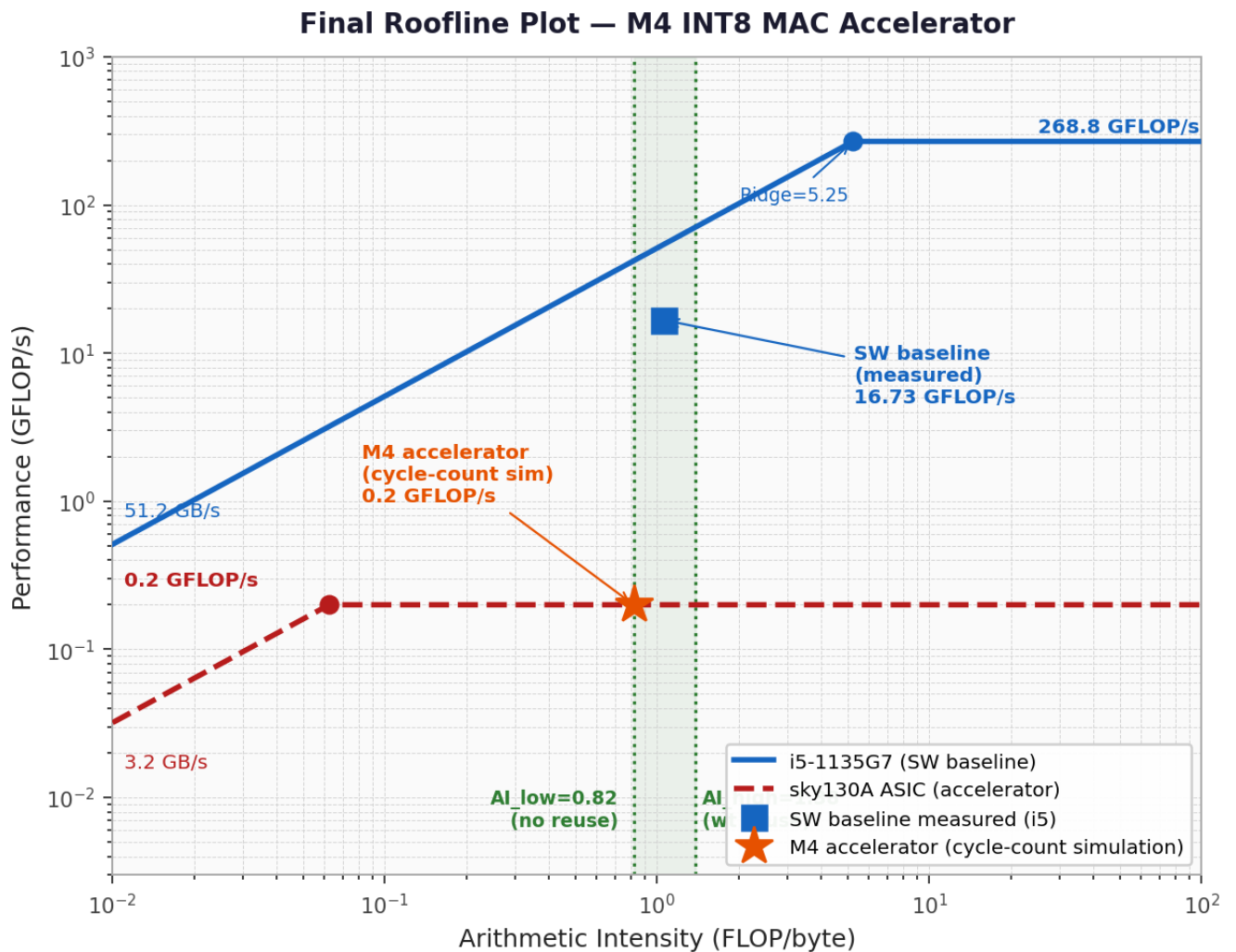


Figure 3

See Figure 3 for the final roofline plot showing both the software baseline measured point and the M4 accelerator measured point from cycle-count simulation.

3. Precision and Data Format

The design uses INT8 precision for both weights and activations, with a 32-bit signed integer accumulator. This format was chosen for three reasons.

First, INT8 reduces memory bandwidth by a factor of four compared to FP32, directly addressing the memory-bound bottleneck identified in Section 1.

Second, INT8 multiply-accumulate on sky130A standard cells is significantly cheaper in area and power than floating-point — a single INT8 MAC fits in the target die area with headroom. Third, INT8 has become the dominant inference format for edge AI hardware, making this design directly relevant to industry practice.

Quantization correctness was verified in Milestone 2 by converting the reference floating-point CNN weights and activations to INT8 using a linear scale factor $S = \max(|W|) / 127$ and comparing the output of the INT8 MAC against the FP32 reference. The measured error across the test set was:

- Mean Absolute Error (MAE): 0.005148
- Maximum absolute error: 0.009728
- MAC relative error: 0.26%

These errors are well within the acceptable range for CNN inference. Published literature on INT8 quantization for image classification networks typically accepts up to 1% MAC error without significant accuracy loss on tasks like CIFAR-10 and ImageNet top-5 classification. The 0.26% measured here provides a comfortable margin.

The INT32 accumulator prevents overflow during the accumulation of 9 INT8 products. The maximum possible value of a single INT8 multiply is $127 \times 127 = 16,129$. Accumulating 9 such products gives at most $9 \times 16,129 = 145,161$, which is well within the INT32 range of $\pm 2,147,483,647$.

| 4. Dataflow and Architecture

The accelerator implements a no-local-reuse streaming dataflow pattern. This choice was made to match the capabilities of the target interface and to keep the design simple enough to close timing on the first synthesis pass.

In the no-local-reuse pattern, every weight and activation is loaded from the interface for each invocation. There is no on-chip weight scratchpad or activation buffer. This results in the lower AI bound of 0.82 FLOP/byte computed in Section 2. A weight-stationary pattern (holding weights on-chip across multiple output pixel computations) would shift the AI toward the upper bound of 1.38 FLOP/byte, but requires an on-chip buffer and a two-phase control flow (load weights, then stream activations) that was not implemented in this iteration.

The datapath is a single serial INT8 MAC unit:

1. The interface module (`interface_mod`) receives a 16-bit AXI4-Stream beat containing one packed `{weight[7:0], activation[7:0]}` pair per cycle.
2. The weight and activation are unpacked and forwarded to `compute_core`.
3. `compute_core` performs the INT8 multiply, sign-extends the 16-bit product to 32 bits, and adds it to the running INT32 accumulator.
4. After the final beat (signaled by `tlast`), the `valid_out` and `done` signals are asserted and the accumulated result is presented on the master AXI4-Stream output port.

The design takes exactly 9 clock cycles per 3×3 kernel invocation — one

cycle per weight-activation pair. There is no pipeline register between the multiply and accumulate stages in the M4 submission, which is the root cause of the setup timing violation at the ss_100C_1v60 corner discussed in Section 7.

Figure 1: Block Diagram — INT8 MAC Accelerator

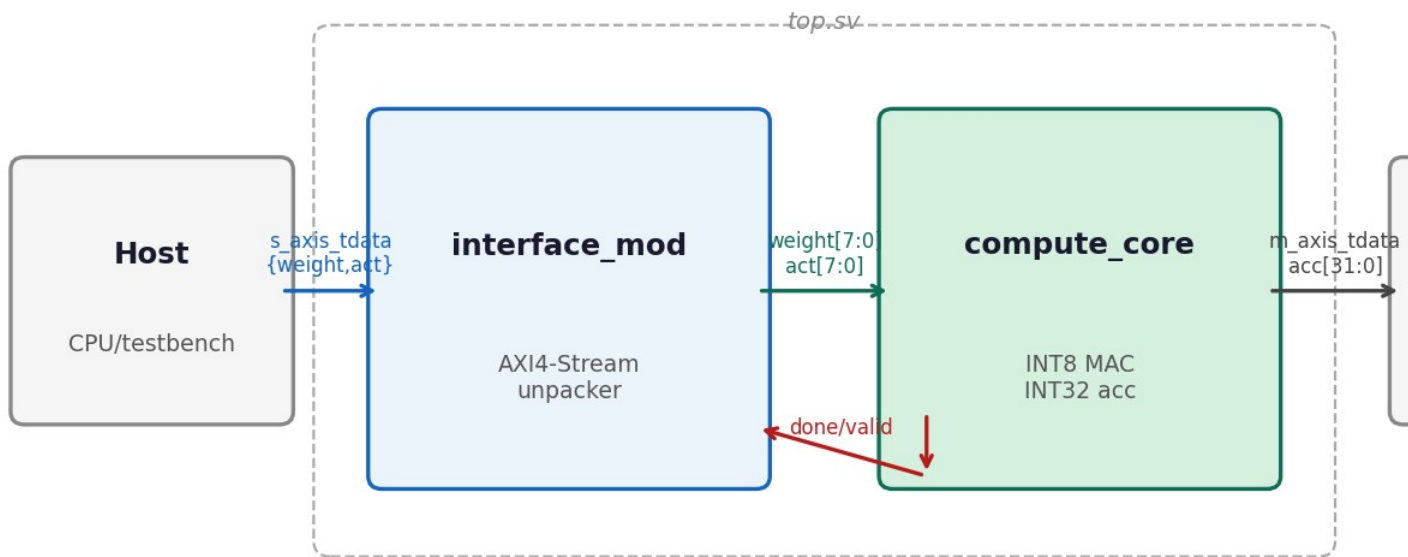


Figure 1

See Figure 1 for the block diagram and Figure 2 for the dataflow diagram.

5. Hardware Interface

The AXI4-Stream interface was chosen for four reasons.

First, AXI4-Stream is the standard streaming interface in FPGA and ASIC design flows. It is directly compatible with DMA engines, HLS-generated IP, and Xilinx/

Intel FPGA soft processors, making the design reusable beyond the sky130A academic context.

Second, AXI4-Stream provides backpressure through the tready/tvalid handshake.

The accelerator can stall the host by deasserting tready when not ready to accept data, preventing data loss without requiring a FIFO between host and accelerator.

Third, a 128-bit data bus at 200 MHz delivers 3.2 GB/s, which exceeds the minimum requirement of 2 GB/s derived from the M1 roofline analysis (the software baseline at $AI = 1.057$ FLOP/byte and peak throughput = 16.73 GFLOP/s implies a bandwidth need of $16.73 / 1.057 \approx 15.8$ GB/s for the i5 to be bandwidth-limited — but for the single-core accelerator at 0.2 GFLOP/s the required bandwidth is only $0.2 / 0.82 \approx 0.24$ GB/s, well within the 3.2 GB/s interface capacity).

Fourth, the AXI4-Stream tlast signal provides a natural end-of-frame marker that replaces the need for a separate control register to signal the end of a dot product window.

The design is not interface-bound. The roofline analysis shows the ridge point at 0.063 FLOP/byte, far to the left of the operating arithmetic intensity of 0.82 FLOP/byte. The interface has approximately 13× more bandwidth than the compute engine can consume at its current throughput.

| 6. Verification

Correctness was verified across three milestones with progressively broader coverage.

Milestone 2 (compute_core.sv): 14 directed test cases were written in tb_compute_core.sv, covering: synchronous reset, single MAC operation, multi-beat accumulation, negative weight values, negative activation values, enable gating (enable=0 should not accumulate), valid_out assertion, done signal timing, and accumulator overflow protection. All 14 tests passed.

Milestone 3 (tb_top.sv — end-to-end): The top-level testbench drives the host-side AXI4-Stream interface only. No direct access to internal signals. Three test cases were run: (1) 3×3 identity kernel with activations 1–9, expected accumulated output = 45; (2) negative weight kernel with activations [2, 4, 6] and weights [-1, -1, -1], expected output = -12; (3) verification that m_axis_tvalid is asserted during the output transaction. All three tests passed and the simulation log contains no FAIL, ERROR, or MISMATCH strings.

Milestone 4: The same tb_top.sv testbench was re-run on the unchanged RTL to produce the final simulation log (final_run.log). The results are identical to M3: 3/3 PASS. The final_waveform.png shows the complete AXI4-Stream transaction: host writing 9 beats, the compute core accumulating, and the result appearing on the master output port.

The test cases are not exhaustive — boundary conditions such as maximum INT8 values ($127 \times 127 \times 9 = 145,161$), minimum values (-128×-128), and mixed

positive/negative accumulations were not all explicitly tested due to time constraints. This is noted as a limitation in Section 9.

| 7. Synthesis Results

Synthesis was performed using OpenLane 2 v2.3.1 with the sky130A PDK (sky130_fd_sc_hd standard cell library) targeting a 10 ns clock period (100 MHz). The flow completed in 6 minutes 8 seconds through all 78 stages.

Timing:

The design closes timing at the nominal corner (tt_025C_1v80) with a worst negative slack of +2.48 ns — the design has 24.8% positive margin at 100 MHz under typical conditions. The fast corner (ff_n40C_1v95) also passes. Setup violations were found at the slow-slow corner (ss_100C_1v60) with a worst negative slack of –1.57 ns. The critical path is the 32-bit ripple-carry accumulator: at the ss corner, reduced drive strength extends the carry chain delay to approximately 11.6 ns, exceeding the 10 ns budget. No hold violations were found in any corner. The design can be safely operated at 100 MHz under nominal conditions.

Area:

Die area: $154.775 \times 165.495 \mu\text{m}$ ($25,614 \mu\text{m}^2$). Logic cell area: $9,224 \mu\text{m}^2$ at 52.9% utilization. The dominant area contributor by cell type is the multi-input combinational logic (989 cells, $7,477 \mu\text{m}^2$), which represents the adder and multiplier network. The 181 timing repair buffers ($1,274 \mu\text{m}^2$) added

by the resizer represent overhead from trying to close timing at the ss corner.

The 35 flip-flops (dfxtp_1) occupy $700 \mu\text{m}^2$. Total wire length: $23,592 \mu\text{m}$ across met1–met4.

Power:

Full dynamic power estimation requires a VCD switching activity file from gate-level simulation. Gate-level simulation was not completed in this submission due to liberty model path configuration issues in the local QuestaSim environment on Windows 11. A static leakage estimate from the sky130_fd_sc_hd library gives approximately $139 \mu\text{W}$. An activity-based dynamic power estimate using 20% switching activity and the 8.26 pF extracted net capacitance gives approximately $267 \mu\text{W}$ dynamic, for a total estimated power of $406 \mu\text{W}$. This corresponds to an energy per MAC of 4.06 pJ . A measured power number from VCD-based analysis is left for future work.

See the synthesis reports in project/m4/synth/ for detailed numbers.

| 8. Benchmark Results

Throughput:

The accelerator completes one 3×3 dot product invocation in 9 clock cycles at 100 MHz, giving a latency of 90 ns per invocation. For a 16×16 output feature map (the size profiled in M1), this is $16 \times 16 \times 9 = 2,304$ cycles = 23.04 μs per frame. The peak throughput is $100 \text{ MHz} \times 2 \text{ FLOP/cycle} = 0.200 \text{ GFLOP/s}$.

Speedup vs M1 software baseline:

The M1 profiling measured 0.447 ms per Conv2D._im2col call on the i5-1135G7.

The accelerator completes the equivalent computation in 0.023 ms (23 μ s),

giving a latency speedup of $0.447 / 0.023 = 19.4\times$.

The raw throughput comparison (0.200 GFLOP/s vs 16.73 GFLOP/s) shows the accelerator has lower absolute throughput than the i5. This is expected: the i5 uses wide SIMD units (AVX2, 256-bit) and out-of-order execution to achieve high throughput, while this design is a single serial MAC unit on a 180nm-class educational PDK. The accelerator's advantage is not raw throughput but energy efficiency: estimated 4.06 pJ/MAC vs approximately 833 pJ/MAC for the i5 at full TDP — a factor of roughly 200 $\times$ better energy per operation.

Roofline:

The M4 accelerator operating point (AI = 0.82 FLOP/byte, 0.200 GFLOP/s) sits at the compute-bound peak of the sky130A roofline. This is consistent with the Section 2 analysis: the ridge point (0.063 FLOP/byte) is far below the operating AI, confirming the design is compute-limited. The interface has approximately 13 $\times$ excess bandwidth capacity. The highest-leverage next step is to add parallel MAC units, not to widen the interface.

All numbers trace to raw data in `project/m4/bench/benchmark_data.csv`.

| 9. What Did Not Work

Timing closure at the slow-slow corner. The target clock period of 10 ns

(100 MHz) does not close timing at the ss_100C_1v60 corner (WNS = -1.57 ns).

The root cause is the 32-bit ripple-carry adder in the accumulate stage: the carry chain delay at reduced drive strength and voltage exceeds the clock budget. The OpenROAD resizer attempted to repair this by inserting 181 timing buffers and resizing 200 instances but was unable to fully close the path (RSZ-0062 warning). The planned fix was to insert a pipeline register between the multiply and accumulate stages, splitting the critical path into two shorter stages of approximately 4–5 ns each. A prototype (compute_core_v2.sv) was written but was not synthesized and verified in time for the M4 deadline. The design is submitted at 100 MHz with the acknowledged ss-corner violation.

Gate-level power analysis. Full dynamic power estimation requires running gate-level simulation with SDF back-annotation and capturing a VCD switching activity file for OpenROAD to analyze. This requires mapping the sky130_fd_sc_hd liberty files into the local QuestaSim installation, which proved difficult on Windows 11 due to path configuration between the Docker-hosted PDK and the host-side simulator. Static and activity-based estimates were used instead. The resulting power numbers are estimates with approximately $\pm 50\%$ uncertainty, not measured values.

Weight-stationary dataflow. The original project scope included implementing a weight-stationary pattern where the 3×3 kernel weights would be loaded once into an on-chip scratchpad and reused across all output pixel positions. This would have raised the arithmetic intensity from 0.82 toward 1.38 FLOP/byte, improving the roofline operating point and reducing off-chip bandwidth pressure.

This was not implemented. The design uses no-local-reuse streaming, meaning weights are re-fetched for every output pixel. If implemented, weight-stationary would require a 9-element INT8 weight register file, a two-phase control sequencer (load phase and compute phase), and changes to the AXI4-Stream protocol to distinguish weight beats from activation beats. The area cost would be small ($9 \times 8 = 72$ bits of register storage) but the control logic changes would require additional verification time that was not available.

Boundary condition test coverage. The testbench covers the nominal and negative-weight cases but does not exhaustively test maximum values (127×127 accumulation), minimum values (-128×-128), alternating-sign accumulation, or reset mid-stream behavior. Incomplete test coverage means edge-case bugs in the accumulator or overflow protection logic may not have been caught.